SCRIPTING LANGUAGES TRAINING

Author: AnhN

Sep 3rd 2019

Updated Sep.3rd 2025

©2025 The information in this document is the property of Quest Global Services Private Limited and may not be disclosed in any manner to a third party nor used for any purpose other than that for which it is supplied without prior written consent.

Agenda

1. Shell scripting languages: c-shell, awk, sed
2. Tcl
3. Perl

Confidential

Shell scripting languages

- Reference documents for c-shell, awk and sed:

http://www.grymoire.com/Unix/Csh.html

http://www.grymoire.com/Unix/Awk.html

http://www.grymoire.com/Unix/sed.html

Confidential

Shell scripting languages

I. C-shell

1. grep
2. If – else
3. Foreach
4. While
5. Switch
6. Variable Modifiers

Confidential

Shell scripting languages

I. C-shell

1. grep: search a user-defined pattern (**-v: ignore**)

```
7:26pm Tue Mar-13 Command -> ls -l ~/
total 56
-rw-rw-r--. 1 svn_07 users  174 Nov 16 04:52 command.log
-rwxr-xr-x. 1 svn_07 users  972 Nov  6 23:02 create_excel
drwxr-xr-x. 2 svn_07 users 4096 Sep 10  2017 Desktop
drwxr-xr-x. 2 svn_07 users 4096 Dec  2 05:57 Documents
drwxr-xr-x. 2 svn_07 users 4096 Sep 10  2017 Downloads
-rw-r--r--. 1 svn_07 users 2896 Oct 26 01:20 env_var
drwxrwxr-x. 3 svn_07 users 4096 Jan 11 07:40 ICC
drwxr-xr-x. 2 svn_07 users 4096 Sep 10  2017 Music
drwxrwxr-x. 5 svn_07 users 4096 Jan 25 07:56 my_scripts
drwxr-xr-x. 2 svn_07 users 4096 Sep 10  2017 Pictures
drwxr-xr-x. 2 svn_07 users 4096 Sep 10  2017 Public
drwxr-xr-x. 2 svn_07 users 4096 Sep 10  2017 Templates
drwxr-xr-x. 3 svn_07 users 4096 Mar  6 01:40 thinguyen
drwxr-xr-x. 2 svn_07 users 4096 Sep 10  2017 Videos
```

```
7:28pm Tue Mar-13 Command -> ls -l ~/ | grep env
-rw-r--r--. 1 svn_07 users 2896 Oct 26 01:20 env_var
```

```
7:29pm Tue Mar-13 Command -> ls -l ~/ | egrep -v "env|create"
total 56
-rw-rw-r--. 1 svn_07 users  174 Nov 16 04:52 command.log
drwxr-xr-x. 2 svn_07 users 4096 Sep 10  2017 Desktop
drwxr-xr-x. 2 svn_07 users 4096 Dec  2 05:57 Documents
drwxr-xr-x. 2 svn_07 users 4096 Sep 10  2017 Downloads
drwxrwxr-x. 3 svn_07 users 4096 Jan 11 07:40 ICC
drwxr-xr-x. 2 svn_07 users 4096 Sep 10  2017 Music
drwxrwxr-x. 5 svn_07 users 4096 Jan 25 07:56 my_scripts
drwxr-xr-x. 2 svn_07 users 4096 Sep 10  2017 Pictures
drwxr-xr-x. 2 svn_07 users 4096 Sep 10  2017 Public
drwxr-xr-x. 2 svn_07 users 4096 Sep 10  2017 Templates
drwxr-xr-x. 3 svn_07 users 4096 Mar  6 01:40 thinguyen
drwxr-xr-x. 2 svn_07 users 4096 Sep 10  2017 Videos
```


Confidential

Shell scripting languages

I. C-shell

2. If - else: example_1.csh

```
#!/bin/csh -f # -f: ignore alias in .cshrc file
# set a = $<     // <: in ~ >:out : get input from keyboard
set a = 1

  if ($a == 1) then  echo $a
else if ($a == 2) then
   @ a = $a + 1 ;# (mathematical expression)  echo $a
else
  echo $a  endif
```


Confidential

Shell scripting languages

I. C-shell

3. foreach: example_2.csh

```
#!/bin/csh -f
foreach file (`ls -1 ~/`)
    echo $file
end
```


Confidential

Shell scripting languages

I. C-shell

4. while: be careful when using while command, you have to make sure that you're using this command correctly, you might enter infinity loop and server might be hang out

```
#!/bin/csh -f
while (1)
  if (! -e "job_finish") then
    echo "Waiting for job_finish flag…"
    sleep 5
  else
    break
  endif  end
```

Shell scripting languages

I. C-shell

-e means file exists, below is file operators:

Operator	Meaning
-r filename	Returns true, if the user has read access
-w filename	Returns true, if the user has write access
-x filename	Returns true, if the user has execute access
-e filename	Returns true, if the file exists
-o filename	Returns true, if the user owns the file
-z filename	Returns true, if the file is empty
-f filename	Returns true, if the file is a plain file (**only file, not link/shortcut**)
-d filename	Returns true, if the file is a directory

Shell scripting languages

I. C-shell

5. switch: We rarely use this command

```
#!/bin/csh -f
set num = 1;
switch ($num)
  case 0:
    echo $num ; breaksw
  case 10:
    echo $num; breaksw
  default:
    echo "$num is not equal to 10"
endsw
```

Shell scripting languages

I. C-shell

6. Variable Modifiers

In C shell, the path value obtained from a variable can be modified before it is used into a command or expression. Variable modifiers are given after a colon (:) at the end of a variable. The meaning of the modifiers are as follows:

```
:h    returns the directory of a path (aka "head")
:t    returns the filename of a path (aka "tail")
:r    returns the directory and filename without the last  extension (aka "root")
:e    returns the extension of the path (aka "end")
```


Confidential

Shell scripting languages

I. C-shell

6. Variable Modifiers (cont.)

Example:

```
set rpt = 2025_Sep/01_User/AnhN/reports/route_icc.qor ; # report file
echo   Full:  $rpt
echo   Head:  ${rpt:h}
echo   Tail:  ${rpt:t}
echo   Root:  ${rpt:e}
echo   End :  ${rpt:r}
```

Output:

```
Full: 2025_Sep/01_User/AnhN/reports/route_icc.qor
Head: 2025_Sep/01_User/AnhN/reports
Tail: route_icc.qor
Root: qor
End : 2025_Sep/01_User/AnhN/reports/route_icc
```


Confidential

Shell scripting languages

II. Awk (Alfred **A**ho, Peter **W**einberger, and Brian **K**ernighan)

1. Fields and Records
2. How to implement an awk script
3. Awk structure
4. Built-in variables
5. Multi-dimensional array
6. Arithmetic operators
7. Relational operators
8. Logical operators
9. Regular expressions
10. Control flow
11. User-defined functions

Confidential

Shell scripting languages

II. Awk

1. Fields and Records:

+ awk stored line information into a variable called NR (Number of Records), each line is separated into many fields and stored into a variable called NF (Number of Fields).

+ $0: whole line

+ $1: first field

+ $NF: last field

=> By default, each field is separated by **one or more spaces** (including tab(s))

Example:

Line	Hello	Quest	Global	Vietnam
Fields	$1 ($(NF-3))	$2 ($(NF-2))	$3 ($(NF-1))	$4 ($NF)
Whole line	$0			

Confidential

Shell scripting languages

II. Awk

2. How to implement an awk script

+ Using awk by it own style:

```
#!/bin/awk -f
BEGIN {
  sum = 0
}
{
  print $0
  sum = sum + $1
}  END {
  print "sum: " sum
}
```


Confidential

Shell scripting languages

II. Awk

2. How to implement an awk script (cont.)

+ Integrate with c-shell script

```
#!/bin/csh -f
set INPUT = “$1”
awk 'BEGIN { \
  sum = 0 \
} \
{ \
  print $0 \
  sum = sum + $1 \
} \
END { \
  print "sum: " sum \
}' $INPUT
```


Confidential

Shell scripting languages

II. Awk

3. Awk structure

```
BEGIN {
  # This part will be executed first, before the first line is being read
}
{
  # Body, processing each line
}  END {
  # This part will be executed last, after the last line was read.
}
```


Confidential

Shell scripting languages

II. Awk

4. Built-in variables

FS	The Input Field Separator Variable (by default, FS is spaces)
OFS	The Output Field Separator Variable
NF	The Number of Fields Variable
NR	The Number of Records Variable
RS	The Record Separator Variable
ORS	The Output Record Separator Variable
FILENAME	The Current Filename Variable
ARGC	The number of arguments provided at the command line
ARGV	An array that stores the command-line arguments
CONVFMT	The conversion format for numbers. Default value is %.6g

Confidential

Shell scripting languages

II. Awk

5. Multi-dimensional array

```
#!/bin/csh -f

set INPUT = $1  awk '{ \
        array[$1] = $2 \
} \  END { \
  for (i in array) { \
    printf ("%-8s %-2d\n", i, array[i]) \ ; "Check “Format Specifiers"
  } \
}' $INPUT

red      3
orange   1
blue     2
yellow   4
green    5
```


Confidential

Shell scripting languages

II. Awk

6. Arithmetic operators

AWK supports the following arithmetic operators:

+ Addition (+)
+ Subtraction (-)
+ Multiplication (*)
+ Division (/)
+ Module (%)
+ Increment & Decrement operators (++ , --)
    - Pre-increment/decrement
    - Post-increment/decrement
+ Shorthand operators: +=, -=, *=, /=, %=, ^=, **=

Confidential

Shell scripting languages

II. Awk

7. Relational operators

+ Equal to (==)
+ Not equal to (!=)
+ Less than (<)
+ Less than or Equal to (<=)
+ Greater than (>)
+ Greater than or Equal to (>=)

8. Logical operators

+ Logical AND (&&)
+ Logical OR (||)
+ Logical NOT (!)

Confidential

Shell scripting languages

II. Awk

9. Regular expressions

+ Dot (.): Matches any single character except the end of line character

```
echo "cat\nbat\nfun\nfin\nfan" | awk '{if ($0 ~ /f.n/) print}'
```

+ Zero or One Occurrence (?): Matches zero or one occurrence of the preceding character.

```
echo "Colour\nColor" | awk '{ if ($0 ~ /Colou?r/ ) print }'
```

+ Zero or More Occurrence (*): Matches zero or more occurrences of the preceding character

```
echo "ca\ncat\ncatt" | awk '/cat*/'
```

+ One or More Occurrence (+): Matches one or more occurrence of the preceding character

```
echo -e "111\n22\n123\n234\n456\n222"  | awk '/2+/'
```


Confidential

Shell scripting languages

II. Awk

9. Regular expressions (cont.)

+ Start of line (^): Matches in the start of line.

```
echo "This\nThat\nThere\nThose\nThese\nThen" | awk '{if ($0 ~ /^The/) print}'
```

+ End of line ($): Matches in the end of line.

```
echo "This\nThat\nThere\nThose\nThese\nThen" | awk '{if ($0 ~ /e$/) print }'
```

+ Match character set ([]): Match only one out of several characters.

```
 echo "Call\nTall\nBall\nHall" | awk '{if ($0 ~ /[CB]all/) print}'
```

+ Exclusive set ([^]): The carat negates the set of characters in the square brackets

```
 echo "Call\nTall\nBall\nHall" | awk '{if ($0 ~ /[^CB]all/) print}'
```

+ Alteration (|): Allows regular expressions to be logically Ored

```
 echo "Call\nTall\nBall\nHall" | awk '{if ($0 ~ /Tall|Hall/) print}'
```

+ Grouping ( ): Uses for grouping and the character | is used for alternatives

```
   echo "Apple Juice\nApple Pie\nApple Tart\nApple Cake" | awk '/Apple  (Juice|Cake)/'
```


Confidential

Shell scripting languages

II. Awk

10. Control flow

+ if – else (if-else-if)

```
echo 10 | awk '{\
   if ($1 % 2 == 0) { \
     printf ("%d is even number\n", $1) \
   } else { \
     printf ("%d is odd number\n", $1) \
   } \
 }'
```

+ for loop

```
awk 'BEGIN { for (i = 1; i <= 5; ++i) print i }'
```

+ while loop

```
awk 'BEGIN {i = 1; while (i < 6) { print i; ++i } }'
```

+ do-while loop

```
awk 'BEGIN {i = 1; do { print i; ++i } while (i < 6) }'
```


Confidential

Shell scripting languages

II. Awk

10. Control flow (cont.)

+ Break statement

```
awk 'BEGIN {
    sum = 0; for (i = 0; i < 20; ++i) {
        sum += i; if (sum > 50) break; else print "Sum =", sum
    }
}
```

+ Continue statement: Uses inside a loop to skip to the next iteration of the loop

```
awk 'BEGIN {
    for (i = 1; i <= 20; ++i) {
        if (i % 2 == 0) print i ; else continue
    }
}'
```


Confidential

Shell scripting languages

II. Awk

10. Control flow (cont.)

+ Exit statement: Use to stop the execution of the script

```
awk 'BEGIN {
    sum = 0; for (i = 0; i < 20; ++i) {
        sum += i; if (sum > 50) exit(10); else print "Sum =", sum
    }
}'

echo $? => 10 (returned error code)
```


Confidential

Shell scripting languages

II. Awk

11. User-defined functions

```
function find_min(num1, num2) {  if (num1 < num2)
return num1
return num2
}
function find_max(num1, num2) {  if (num1 > num2)
return num1  return num2
}
function main(num1, num2) {  result = find_min(10, 20)  print "Minimum =", result  result =
find_max(10, 20)  print "Maximum =", result
}  BEGIN {
main(10, 20)
}
```


Confidential

Shell scripting languages

II. Awk

12. Some useful options with awk (-F ; -v)

-F "fs": specify the field separator with "fs"

-v var=value: assign the variable from outside into awk command

Example:

```
set value = "toys"  cat file1.txt
cats dogs, fish, birds, ducks.  awk '{print $1,$2,$3}' file1.txt
cats dogs, fish,
awk -F "," '{print $1,$2,$3}' file1.txt
cats dogs     fish    birds
awk -v var=$value '{print $1,$2,$3,var}' file1.txt
cats dogs, fish, birds, toys
```


Confidential

Shell scripting languages

III. Sed (**S**tream **ED**itor)

1. Work flow: Read -> Stream -> Display

+ Read: SED reads a line from the input stream (file, pipe, or stdin) and stores it in its internal buffer called pattern buffer.

+ Execute: All SED commands are applied sequentially on the pattern buffer. By default, SED commands are applied on all lines (globally) unless line addressing is specified.

+ Display: Send the (modified) contents to the output stream. After sending the data, the pattern buffer will be empty.

+ The above process repeats until the file is exhausted.

Confidential

Shell scripting languages

III. Sed (Stream EDitor)

2. Basic syntax

```
sed [-n] [-e] 'command(s)' files
sed [-n] -f script files

-n: Default printing of pattern buffer
-e: Next argument is an editing command
-i: replace in file
```

3. Example

```
echo "welcome new internship students" | sed -e "s/welcome/Welcome/g"

-e "s/students/engineers/g"

=> Output: Welcome new internship engineers

echo "/home/u/AnhN/working_dir" | sed "s/\(.*\)\/\(.*\)/\1/"
=> /home/u/AnhN
echo "/home/u/AnhN/working_dir" | sed "s/\(.*\)\/\(.*\)/\2/"
=> working_dir
```

TCL-Tool Command Language

©2025 The information in this document is the property of Quest Global Services Private Limited and may not be disclosed in any manner to a third party nor used for any purpose other than that for which it is supplied without prior written consent.

TCL-Tool Command Language

AGENDA

1. Basic Command
2. Control Structure
3. Procedures
4. String
5. List
6. Array
7. File system

Confidential

TCL - 1. Basic commands

- **Print to screen (puts)**

```
% puts "Hello Synapse ^_^!"
>> Hello Synapse ^_^!
```

- **Assignment (set / setenv)**

```
% set Income 15000
>> 15000
% puts "Income is $Income"
>> Income is 15000
```

(using '$' to get the value of a variable)

- **Mathematical expressions (expr)**

```
% set a 10.0
>> 10.0
% expr $a + 5
>> 15.0
% expr int($a/3)
>> 3
```

TCL - 1. Basic commands

- **Unset a variable (unset) or unset environment variable (unsetenv)**

```
% set a 10.0
>> 10.0
% expr $a + 5  15.0
% expr int($a/3)   3
% unset a
% puts $a
can't read "a": no such variable
```

- **Check whether the named variable has been defined (info)**

```
% if {![info exists a]} {set a 10} else {set a 9}; puts $a  10
```


Confidential

TCL - 1. Basic commands

- Increments the value of a variable (incr)

Increments the value stored in the variable whose name is varName. The value of the variable must be an integer. If increment is supplied then its value (which must be an integer) is added to the value of variable varName; otherwise 1 is added to varName. The new value is stored as a decimal string in variable varName and also returned as result.

+ incr x	: Increment x by 1
+ incr x 10	: Increment x by 10
+ incr x $y	: Increment x by value of y

```
% set b 7
7
% incr a $b
30
% set c 7.5
7.5
% incr c
expected integer but got "7.5"
```


Confidential

TCL – 2. Control Structure

- **If … elseif … else**

```
set income 32000
  if {$income > 30000} {  puts "$income -- high"
  } elseif {$income > 20000} {  puts "$income -- middle"
} else {
   puts "$income -- low"
}
```

Result: 32000 -- high

- **while loops**

```
set i 0
while {$i < 10} {
   puts "I am at count $i"  incr i
}
```

Result:

```
% while { $i < 10 } { puts "I am at count $i" ; incr i }
I am at count 0
I am at count 1
I am at count 2
I am at count 3
I am at count 4
I am at count 5
I am at count 6
I am at count 7
I am at count 8
I am at count 9
%
```

TCL – 2. Control Structure

- **for loops**

```
for {set i 0} {$i < 10} {incr i} {
  puts "I am at count $i and going up"
}
for {set i 10} {$i > 0} {incr i -1} {
  puts "I am at count $i and going down"
}
```

- **foreach loops**

```
set list_colour {red orange yellow green blue purple}
foreach colour $list_colour {
  puts $colour
}
```

```
% for {set i 0} {$i < 10} {incr i} {  puts "I am at count $i and going up" }
I am at count 0 and going up
I am at count 1 and going up
I am at count 2 and going up
I am at count 3 and going up
I am at count 4 and going up
I am at count 5 and going up
I am at count 6 and going up
I am at count 7 and going up
I am at count 8 and going up
I am at count 9 and going up
```

```
% set list_colour {red orange yellow green blue purple}
  foreach colour  $list_colour {
      puts $colour }
red orange yellow green blue purple
% red
orange
yellow
green
blue
purple
%
```


Confidential

TCL – 3. Procedures

- **Create your own procedure**

```
proc foo {a b c} {
    return [expr $a * $b - $c]
}
puts [expr [foo 2 3 4] + 5]
```

Result:

```
I'm in the bar procedure
% proc foo {a b c} {
    return [expr $a * $b - $c]
}
puts [expr [foo 2 3 4] + 5]
% 7
```

```
proc bar { } {
    puts "I'm in the bar procedure"
}
bar
```

Result:

```
% proc bar { } {
    puts "I'm in the bar procedure"
}
bar
% I'm in the bar procedure
```


Confidential

TCL – 4. String

- **General syntax**

```
string operation stringvalue otherargs
```

`string compare str1 str2`	Compare 2 strings lexicographically, return 0 if equal, -1 if str1 sorts before str2, else 1
`string first str1 str2`	Return the index in str2 of the first occurrence of str, or -1 if str1 is not found.
`string index string` ***index***	Return the character at the specified ***index***
`string last str1 str2`	Return the index in str2 of the last occurrence of str1, or – if str1 is not found
`string length string`	Return the number of character in string
`string match pattern str`	Return 1 if str matches the pattern, else 0
`string range str i j`	Return the range of characters in str from i to j
`string tolower string`	Return string in lower case
`string toupper string`	Return string in upper case

Confidential

TCL – 4. String

- **General syntax**

`string trim string ?chars?`	Trim the characters in chars from both ends of string. chars defaults to whitespace
`string trimleft string ?chars?`	Trim the characters in chars from the beginning of string. chars defaults to whitespace
`string trimright string ?chars?`	Trim the characters in chars from the end of string. chars defaults to whitespace
`string wordend str ix`	Return the index in str of the character after the word containing the character at index ix
`string wordstart str ix`	Return the index in str of first character in the word containing the character at index ix

TCL – 4. String

- **Regular Expressions**

```
.           Matches any character
*           Matches zere or more
+           Matches one or more
?           Matches zero or one
()          Groups a sub-pattern. The repetition and alteration operators
   apply              to the whole proceeding sub-pattern
|           Alternation
[]          Delemit a set of characters. Range are specified as [x-y]
            If the first character in the set is ^, then there is a match if
            the remaining characters in the set are not present.
^           Anchor the pattern to the beginning of the string. Only when first
$           Anchor the pattern to the end of the string. Only when last
```

TCL – 4. String

- **The regexp command**

The regexp command provides direct access to the regular expression matcher. Syntax:

```
regexp ?flag? pattern string ?match sub1 sub2...?
```

The return value is 1 if some part of the string matches the pattern, it is 0 other wise Example:

```
set a "/home/AnhN/scripts/fresher_traning/tcl"  regexp
{(.*)/(.*)/(.*)} $a result part1 part2 part3
1 => 1 means matched
```

Output:

```
puts $result
/home/AnhN/scripts/fresher_traning/tcl  puts $part1
/home/AnhN/scripts  puts $part2
fresher_traning  puts $part3
tcl
```

TCL – 4. String

- **The regsub command**

The regsub command is used to do string substitution based on pattern matching. Syntax:

```
regsub ?switches? pattern string subspec varname
```

The regsub command returns the number of matches and replacements, or 0 if there was no match. Regsub copied *string* to *varname*, replacing occurrences of *pattern* with the substitution specified by *subspec*

Example:

```
set a "/home/AnhN/scripts/fresher_traning/tcl"  regsub -all {/}
$a {_} b
5 => 5 is the returns numbers of matches and replacements  regsub
{([^\.]*)\.c} file.c {this is new output: \1.tcl} output
```

Output:

```
puts $b
_home_AnhN_scripts_fresher_traning_tcl  puts $output
this is new output: file.tcl
```


Confidential

TCL – 4. String

- **The glob command**

The glob command returns names of file that match patterns

Syntax:

```
glob
  -directory directory
  -join
  -nocomplain
  -path pathPrefix
  -tails
  -types typeList
```

Example:

```
set files [glob -nocomplain -directory ./ -types {f d} -tails *]
foreach file $files {puts $file}
```

Note:

```
-nocomplain: if we used this flag, glob will return an empty list in stead of  error if no files
match
```

TCL – 5. List

- **Create list**
  - **Many ways to create a list**

```
 set myList [list a b c]  set myList "a b c"
set myList {a b c}
set myList [list $a $b $c]  set myList {$a $b $c}

set myList [list a   b   c]
set myList "a   b   c"
set myList {{a b c} {d e f} {g h i}}
set s Hello
puts "The length of $s is [string length $s]."
=> The length of Hello is 5.
```


Confidential

TCL – 5. List

- **List operations : lindex**

  - `lindex is used to retrieve elements from list`
  - `lindex list ?index1 ?index2 …`
  - `lindex {a b c} end` → *c*
  - `lindex {a b c} end-1` → *b*
  - `lindex {{{a b} {c d}} {{e f} {g h}}} 1 1 0` → *g*
  - `set idx 1`
  - `lindex {a b c d e f} $idx+2` → *d*
  - `set idx 3`
  - `lindex {a b c d e f} $idx+2` → *f*

Confidential

TCL – 5. List

- **List operations: llength**
  - Treats *list* as a list and returns a decimal string giving the number of elements in it.

```
llength {a b c d e}          → 5
llength {a b c}              → 3
llength {}                   → 0
```

  - Elements are not guaranteed to be exactly words in a dictionary sense of course, especially when quoting is used:

```
llength {a b {c d} e}        → 4
llength {a b { } c d e}      → 6
```


Confidential

TCL – 5. List

- **List operations: lappend**

```
set var   1                    → 1
lappend   var 2                → 1 2
lappend   var 3 4 5            → 1 2   3 4 5
```

- **List operations: linsert**

```
set      oldList   {the fox     jumps over   the dog}
set      midList   [linsert     $oldList 1   quick]
set newList [linsert $midList end-1 lazy]
set newerList [linsert [linsert $oldList end-1 quick] 1 lazy]
```

Result:

```
% set oldList {the fox jumps over the dog}
      set midList [linsert $oldList 1 quick]
      set newList [linsert $midList end-1 lazy]
      set newerList [linsert [linsert $oldList end-1 quick] 1 lazy]
the fox jumps over the dog
% the quick fox jumps over the dog
% the quick fox jumps over the lazy dog
% the lazy fox jumps over the quick dog
%
```


Confidential

TCL – 5. List

- **Array operations**

Associative arrays (string as index)

```
set color(rose) red  set color(sky) blue  set color(medal)
gold  set color(leaves) green
set color(blackboard) black
```

```
puts [array exists color]          → 1
```

(tests if an array with the name "color" exists)

```
puts [array exists colour]         → 0
```

```
puts [array names color] ->
```

(returns a list of the index strings)

→ Leaves blackboard rose medal sky

```
foreach item [array names color] {
  puts "$item is $color($item)"
}
```

```
leaves is green
blackboard is black
rose is red
medal is gold
sky is blue
```

TCL – 6. File system

- **File command options**

`file atime name`	Return access time as a decimal string
`file dirname name`	Return parent directory of file *name*
`file executable name`	Return 1 if *name* has execute permission, else 0
`file exists name`	Return 1 if *name* exists, else 0
`file extension name`	Return the part of *name* from the last dot '.' to the end
`file isdirectory name`	Return 1 if *name* is a directory, else 0
`file isfile name`	Return 1 if *name* is not a directory, symbolic link, or device, else 0
`file lstat name var`	Place stat results about the link *name* into *var*
`file mtime name`	Return modify time of *name* as a decimal string
`file owned name`	Return 1 if current user owns the file *name*, else 0

Confidential

TCL – 6. File system

- **File command options**

`file readable name`	Return 1 if *name* has read permission, else 0
`file readlink name`	Return the contents of the symbolic link *name*
`file rootname name`	Return all but the extension (‘.’ and onwards) of *name*
`file size name`	Return the number of bytes in *name*
`file stat name var`	Place stat results about *name* into array *var*
	The elments defined for *var* are: atime, ctime, dev, gid, ino, mode, mtime, nlink, size, type, and uid.
`file tail name`	Return all characters after last ‘/’ in *name*
`file type name`	Return type identifier, which is one of: file, directory, characterSpecial, blockSpecial, fifo, link, or socket
`file writable name`	Return 1 if *name* has write permission, else 0

Confidential

TCL – 6. File system

- **Input/Output**

  Tcl commands used for file accesss:

`open what ?access? ?permissions?`	`Open a file or pipeline.`
`puts ?-nonewline? ?stream? String`	`Write a string.`
`gets stream ?varname?`	`Read a line.`
`read ?-nonewline? stream?numBytes?`	`Read bytes.`
`tell stream`	`Return the seek offset.`
`seek stream offset ?origin?`	`Set the seek offset. origin is one of start, current, or end`
`eof stream`	`Query end-of-file status.`
`flush stream`	`Write out buffers of a str`
`close stream`	`Close an I/O stream.`

TCL – 6. File system

- **Input/Output**

  “open” access arguments:

```
r       Open for reading. The file must exist.
r+      Open for reading and writing. The file must exist.
w       Open for writing. Truncate if it exists.
        Create if it does not exist.
w+      Open for reading and writing. Truncate or create.
a       Open for writing. The file must exist. Data is appended to the file.
a+      Open for reading and writing. File must exist. Data is appended.
```


Confidential

TCL – 6. File system

- **Input/Output**

Example: read input file and write output file.

```
#!/usr/bin/tclsh

set input [open "input.txt" r]
set output [open "output.txt" w]

while {[gets $input line] >= 0 } {
        puts $line
        puts $output "output: $line"
}

close $input
close $output
```

TCL – 6. File system

- **Input/Output**

Example: To **get input from keyboard**

```
#!/usr/bin/tclsh

puts -nonewline "Enter a number: "
set num [gets stdin]
puts "You have entered: $num"
```


Confidential

PERL
Practical Extraction and Report Language

©2025 The information in this document is the property of Quest Global Services Private Limited and may not be disclosed in any manner to a third party nor used for any purpose other than that for which it is supplied without prior written consent.

PERL - Practical Extraction and Report Language

AGENDA

1. Basic Operator
2. Control Structure
3. List & Array
4. Hash/Associative Array
5. Read/Write a file
6. Subroutine

Confidential

PERL – 1. Basic Operator

- **Assigning values**

  - Variables are assigned values using the equal sign:

```
$string1        = "This is a test"; #(remember to add “;” after each command)
$var1           = 6;
$var2           = 3.14159;
```

  - You can assign operation results, as you would expect:

```
my $var3         = $var2 + $var1; #("my": define local variable)
```

PERL – 1. Basic Operator

Perl operators

- Perl supports the standard mathematical operators +, -, *, /, % (modulus) and ** (exponent).
- Operator order of precedence applies according to standard rules; parentheses group operations.
- Operators can be combined in statements:
```
$num1 = $num2 * ((3 + 8)*$num3/2);
```
- Multiple assignments can be made on one line:
```
$num1 = $num2 = $num3=7;
```

Positive and negative

- Numbers are assumed to be positive unless you specify a negative sign in front:

```
$num1       = 6;   # positive
$num2       = -6;  # negative
$num3       = -(-6);      # positive
```

- You can convert positive to negative any time using the minus sign in front of the variable:

```
$num4 =-$num4;
```


Confidential

PERL – 1. Basic Operator

- **Increment and decrement**

- Like C/C++ and Java, Perl supports autoincrement and autodecrement operators, which increase or decrease a value by one:

`$var1++;` is the same as `$var1 = $var1 + 1;`

and

`$var2--;` is the same as `$var2 = $var2 - 1;`

- **Operators and strings**

- Strings can be used with the "." operator for concatenation:

```
$str1  = "Hello";
$str2  = "World!";
$str3  = $str1 . $str2;  print $str3;
```

=> Would print **"Hello World!"**

- You can also concatenate on output in most cases by specifying the string variables together:

```
print $str1 . $str2;
```


Confidential

PERL – 1. Basic Operator

- **Comparison operators**

Perl supports the standard comparison operators:

>	greater than
<	less than
>=	greater than or equal to
<=	less than or equal to
==	exactly equal to
!=	not equal to

- **Other operators**

Perl supports several other operators:

int	returns the integer portion
cos	cosine
Sin	sine
rand	random number between 0 and argument
length	length of argument
Lc	converts to lowercase
Uc	converts to uppercase

Confidential

PERL – 1. Basic Operator

The if statement

- Perl's if statement is similar to those of other high-level languages:

```
if (condition) {
  do if true
} else {
  do if false
}
```

- The blocks of code can contain many statements. The else and its statements are optional.
- Only one block of code is executed, depending on whether the condition is true or false

Example of if

- A simple if statement is:

```
if ( $num1 > 5 ) {
  print "It is greater than 5.\n";
} else {
  print "It is less than or equal to 5.\n";
}
```

PERL – 2. Control Structure

- **Reading Input**

- The easiest way to get input into a Perl program is to read from the keyboard. To do this, use the <STDIN> structure (the standard input, or STDIN, is the keyboard by default).

- You use <STDIN> to read a value and store it in a variable like this:

```
#!/usr/bin/perl -w

print "Please enter a number: ";  my $num = <STDIN>;
chomp($num);
print "The number you have entered is: $num\n";

Output:
Please enter a number: 10
The number you have entered is: 10
```


Confidential

PERL – 2. Control Structure

- **The chomp operator**

- When reading input from the keyboard, the entire string entered by the user, including the RETURN is saved in the assigned variable
- If you want to eliminate the newline character at the end of the input, use the chomp operator to remove it:

```
$str1=<STDIN>;
chomp $str1;
print "You said " . $str1 . "!\n";
```

without the chomp operator, the print statement would have moved down a line at $str1

PERL – 2. Control Structure

String relationships

- There are relational operators for strings (actually, for non-numbers):

```
eq          equal to
ne          not equal to
gt          greater than
lt          less than
ge          greater than or equal to
le          less than or equal to
```

- Comparisons are left-to-right, using ASCII values

Example of string comparisons

```
$str1 = "Hello";
$str2 = <STDIN>;
chomp $str2;
if ($str1 eq $str2) {
  print "You guessed the string!\n";
} else {
  print "Wrong guess!\n";
}
```


Confidential

PERL – 2. Control Structure

- **The for loop**

```
for ($x = 1; $x <= 10; $x++) {
  print $x . "\n";
}
```

- This program will print the numbers from one to ten in a column and then exit. The initial condition sets the value of $x to 1, each loop increments it by one, and the condition is true until x in incremented to 11.

- **The while loop**

```
$num1 = 6;
while ($num1 < 12) {
  print $num1;
  $num1++;
}
```

- This loop will print the values of $num1 starting at 6 and going through to 11. The loop fails when $num1 is incremented to 12.

Confidential

PERL – 3. List & Array

- Lists are groups of scalars and usually reflect multiple values of the same type of data. There are several ways to represent lists, but the most common is to put the list in parentheses, separated by commas:
("Tim", 1234567, 16.65)

- This list has a string value and two numbers, one an integer and one a float (although you don't care about the numeric types). Each part of this list is a "list element" so there are three list elements here.

PERL – 3. List & Array

Assigning lists

- You can assign a list to a variable the same way as any other value. This creates an array variable containing the list:

```
@list1 = ("Tim", 45, 56.0);
```

- When one list contains another list, you use the same syntax:

```
@list2 = (@list1, "sample", 5.3);
```

- You can combine lists and variables:

```
@list2 = (@list1, "sample", $num1);
```


Confidential

PERL – 3. List & Array

- **Getting elements from an array**

- You can retrieve list (array) elements by using their element index value. All list elements are numbered, starting from zero, left to right. To access an array element, use the element number in square brackets:

```
@test=(0,1,2,3,4,5);
print $test[3];
```

- This will display the fourth element (because of zero-origin) which will be 3
- The **length of list: $length = @test;** The result of length variable is 6.

Confidential

PERL – 3. List & Array

- **The for loop and arrays**

- You already know you can access a list element using its index number, such as $list[4];
- Often you will have to move along an entire array, working with each element in the list. You can do this with a for loop:

```
for ($x = 1; $x < @list; $x++) {

  print $list[$x];
}
```

- The @list in the for condition evaluates to a scalar, the length of the list

Confidential

PERL – 3. List & Array

- **The for loop and arrays**

- Other functions to work with array: push, pop, unshift, shift

```
#!/usr/bin/perl -w

# create a simple array
@coins = ("Quarter","Dime","Nickel");
print "1. \@coins  = @coins\n";

# add one element at the end of the array
push(@coins, "Penny");
print "2. \@coins  = @coins\n";

# add one element at the beginning of the array
unshift(@coins, "Dollar");
print "3. \@coins  = @coins\n";

# remove one element from the last of the array.
pop(@coins);
print "4. \@coins  = @coins\n";

# remove one element from the beginning of the array.
shift(@coins);
print "5. \@coins  = @coins\n";
```

- Output:

```
1. @coins = Quarter Dime Nickel
2. @coins = Quarter Dime Nickel Penny
3. @coins = Dollar Quarter Dime Nickel Penny
4. @coins = Dollar Quarter Dime Nickel
5. @coins = Quarter Dime Nickel
```


Confidential

PERL – 3. List & Array

Some commands:

- **sort**

Example: @list = (1, 5, 2, 6, 7);

- Sorting a numberically list by **ascending**:

```
@sorted1 = sort {$a <=> $b} @list;
    --> @sorted1 = (1, 2, 5, 6, 7);
```

- Sorting a numberically list by **descending**:

```
@sorted2 = sort {$b <=> $a} @list;
    --> @sorted2 = (7, 6, 5, 2, 1);
```

- Sorting a alphabet list: `@sorted3 = sort {$b cmp $a} @list;`

PERL – 4. Hash/Associative Array

- **Define a hash/associate array**

- We use “%” character to define a hash:

```
my %color;    #local variable
%NAME;        #global variable
```

- Creating hash:

```
my %component = (
        "monitor", 2,
        "keyboard", 3,
        "mouse", 4,
        "HDD", 5,
);

my %component = (
        "monitor" => 2,
        "keyboard" => 3,
        "mouse" => 4,
        "HDD" => 5,
);
```


Confidential

PERL – 4. Hash/Associative Array

- **Extracting key and value**

- Get all information:

```
foreach my $type (keys(%component)) {
  print "$type $component{$type}\n";
}
```

- Get specific information:

```
print "$component{keyboard}\n";
```

- **Getting hash size**

```
@key = keys %component;
$size = @key;
print "1: hash size: $size\n";

@values = values %component;
$size = @values;
print "2: hash size: $size\n";
```

PERL – 4. Hash/Associative Array

- **Adding and Deleting array elements**

- Adding an element into hash:

```
$component{"USB"} = 6;
```

- Deleting an element from hash:

```
delete ($component{"HDD"});
```

- **Checking for existence**

```
if (exists $component{"DVD"}) {
  print "DVD: $component{DVD}\n";
} else {
  print "Key \"DVD\"does not exist\n";
}
```

PERL – 5. File operations

- **Opening a file**

```
open (filevar, filename);
filevar : The name you want to use in your program to refer to the file  filename: The location
of your input file
```

- The filevar is also known as file variable (or file handle): It can be any sequence of letters, digits and unerscores, as long as the first character is a letter.

- The available file modes are: read (default open mode), write and append.

  + To open a file as read mode: Just use default syntax

```
open (INPUT, "/home/AnhN/input.txt");
```

  + To open a file as write mode: Use ">" character before *filename*

```
open (OUTPUT, ">/home/AnhN/output.txt");
```

  + To open a file as append mode: Use ">>" character before *filename*

```
open (OUTPUT, ">>/home/AnhN/output.txt");
```

PERL – 5. File operations

- **Reading from a file**

- There are 2 ways to read information from a file:

+ Assign filevar into an array, then access it by for/foreach command:

```
open (INPUT, ${input_file});
@INPUTS = <INPUT>;
for (my $i = 0; $i < @INPUTS; $i++) {
     chomp($INPUTS[$i]);  print
     "$INPUTS[$i]\n";
}
or
foreach my $line (@INPUT) {
     chomp($line);
     print "$line\n";
}
```

+ Using while loop to access each line:

```
open (INPUT, ${input_file});
while (my $line = <INPUT>) {
     chomp($line);
     print "$line\n";
}
close INPUT;
```


Confidential

PERL – 5. File operations

- **Writing into a file**

```
open (OUTPUT, ">/home/AnhN/output.txt");
while (my $line = <INPUT>) {
     chomp($line);
     print OUTPUT "$line\n";
}
close (OUTPUT);
```


Confidential

PERL – 5. File operations

- **File operators**

Operator	Description
-b	Is name a block device?
-c	Is name a character device?
-d	Is name a directory?
-e	Does name exist?
-f	Is name an ordinary file?
-g	Does name have its setgid bit set?
-k	Does name have its "sticky bit" set?
-l	Is name a symbolic link?
-o	Is name owned by the user?
-p	Is name a named pipe?
-r	Is name a readable file?
-s	Is name a non-empty file?
-t	Does name represent a terminal?
-u	Does name have its setuid bit set?
-w	Is name a writable file?
-x	Is name an executable file?

Confidential

PERL – 5. File operations

- **File operators**

-x	Is name an executable file?
-z	Is name an empty file?
-A	How long since name accessed?
-B	Is name a binary file?
-C	How long since name's inode accessed?
-M	How long since name modified?
-O	Is name owned by the "real user" only?*
-R	Is name readable by the "real user" only?*
-S	Is name a socket?
-T	Is name a text file?
-W	Is name writable by the "real user" only?*
-X	Is name executable by the "real user" only?*

Confidential

PERL – 6. Subroutine

- **Definition**

- In Perl, a subroutine is a separate body of code designed to perform a particular task. A Perl program executes this body of code by calling or invoking the subroutine; the act of invoking a subroutine is called a subroutine invocation.

```
#!/usr/bin/perl -w

my $num1 = 1;
my $num2 = 2;
my $sum = &sum($num1, $num2);
print "sum: $sum\n";

sub sum {
     my ($num1, $num2) = @_;
     my $sum = $num1 + $num2;
     return $sum;
}
```

PERL – 7. Generating Excel File

- **Definition**

-In Perl, you can summarize the report and information with excel format. It has 2 ways to write the script to generate excel files:

1) Perl → CSV file → Read CSV file on Excel

- Advantage: easy to write the Perl script

- Disadvantage: hard to control the format of excel file such as color of characters, font size, width of column, merge column, ...

2) Perl → Excel file

- Advantage: easy to control the format of excel file

- Disadvantage: hard to write the Perl script

		Timing		
		ICC2		
		post_cts	route	route_opt
WNS/TNS/NVP		func_ssgnp0p72v125c_rcworst_CCworst		
Setup	R2R	-0.2857/-31.4450/1508	-0.8007/-37169.1697/264019	-0.4731/-3882.0141/39188
Hold	R2R	0/0/0	0/0/0	0/0/0
WNS/TNS/NVP		func_ffgnp0p88vn40c_cbest_CCbest		
Setup	R2R	0/0/0	0/0/0	0/0/0
Hold	R2R	-0.4818/-18.2507/2863	-0.3508/-439.9779/26282	-0.0779/-0.6950/400
WNS/TNS/NVP		scan_capture_ssgnp0p72v125c_rcworst_CCworst		
Setup	R2R	-0.0044/-0.0072/2	-0.4398/-3.3747/26	-0.0558/-0.0558/1
Hold	R2R	0/0/0	0/0/0	0/0/0

PERL – 7. Generating Excel File

1. Perl → CSV file → Read CSV file on Excel

-CSV (comma-separated value) file is a simple text file which information is separated by commas, and is common in spreadsheet apps.

- Comma can be replaced by semicolon (;), tab, space, ...

- Output CSV file:

```
1,blue,green,red,yellow  2,tiger,cat,lion,shark
3,computer,speaker,mouse,micro
```

- Read CSV file on Excel:
  - soffice file_name.csv &

	A	B	C	D	E	F
1	1	blue	green	red	yellow	
2	2	tiger	cat	lion	shark	
3	3	computer	speaker	mouse	micro	
4						
5						

Confidential

PERL – 7. Generating Excel File

2. Perl → Excel file

- Perl has few modules that can deal with excel files. It is able to handle all type of spreadsheets.
- Need to define module Spreadsheet::WriteExcel and excel lib before use excel commands.

```
#!/usr/bin/perl
use lib "/nfs/site/home/AnhN/Downloads/Spreadsheet_package/Spreadsheet-WriteExcel-2.37/lib";
use Spreadsheet::WriteExcel;


my $workbook = Spreadsheet::WriteExcel->new("$excel_name.xls");
my $border_for_title_left   = $workbook->add_format( font => 'Calibri', size => 12,align => 'left',  border => 7, bold =>
1,bg_color => 'cyan' );
my $Summary = $workbook→add_worksheet("Summary");
$Oinformation->write(3,0,"Utilization",$border_for_title_left);
$OInformation->set_column(0, 0, 15);
```

- Excel Commands support page: https://metacpan.org/pod/Excel::Writer::XLSX

Thank you